Lecture 05.09: Static Methods (@staticmethod)

📌 Executive Summary

The @staticmethod decorator, decoupling from instance and class state, utility functions, and when to use static vs class vs instance methods.

🔬 Theoretical Foundations & Principles

1. Static Method Mechanics

A `@staticmethod` is a plain function scoped within a class namespace for organizational cohesion. It receives neither `self` nor `cls` automatically:

```
class MathUtils:
    @staticmethod
    def is_even(n: int) -> bool:
        return (n & 1) == 0
```

2. Method Decision Matrix

Method Type	First Parameter	Accesses Instance?	Accesses Class?	Typical Use Case
Instance	`self`	Yes	Yes (via `type(self)`)	Modifying object state
Class	`cls`	No	Yes	Alternative constructors, class state
Static	None	No	No	Pure utility helpers, validation

💻 Production Implementation & Code

```
class SecurityValidator:
    MIN_PASSWORD_LENGTH = 8

    @staticmethod
    def is_valid_password(pwd: str) -> bool:
        has_min_len = len(pwd) >= SecurityValidator.MIN_PASSWORD_LENGTH
        has_upper = any(c.isupper() for c in pwd)
        has_digit = any(c.isdigit() for c in pwd)
        return has_min_len and has_upper and has_digit

print("Password 'Secret123':", SecurityValidator.is_valid_password("Secret123"))
print("Password 'weak':", SecurityValidator.is_valid_password("weak"))
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.